



google / adk-python ▾



Code

Issues

347

Pull requests

247

Discussions

Actions

Projects

Security and quality

Insights

New issue



Raw event.content assigned to child.output without transformation in _reconstruct_node_states during node re-run, for the nodes with message_as_input = True #5553

Closed

Bug

Assignees



Labels

request clarification

v2

workflow



samarth1224 opened on Apr 30

Contributor



Describe the Bug:

I encountered bug during the execution of following code:-

```
analyzer = Agent(
    name="analyzer_agent",
    model="gemini-3.1-flash-lite-preview",
    output_schema=Action,
    instruction=analyzer_prompt,
)

@node
async def main_workflow(ctx: Context):
    report = await ctx.run_node(check_network_status_node)
    if not report:
        yield "network Issue"
        return

    agent_output = await ctx.run_node(analyzer, report) #Analyzer is LLMagent
    agent_output = Action.model_validate(agent_output)

    while agent_output.output_type == 'request_approval':

        result = await ctx.run_node(handle_process, agent_output)
        # main_workflow node start re-running after here
        # And node subsequent to this do not run right after.
        if result:
            agent_output = await ctx.run_node(analyzer,result)
            agent_output = Action.model_validate(agent_output)
        else:
```



```

        print('No result')

    yield agent_output

root_agent = Workflow(
    name="network_analyzer",
    edges=[("START", main_workflow)]
)

```

The await `ctx.run_node(analyzer, report)` is producing types.Content object. instead of dictionary object expected, during the re-run/resuming of the node(main_workflow re-runs, therefore. all the child of it re-runs)

The fresh execution of the nodes runs fine and produces the excepted output. The issue evolves when the parent node(main_workflow) is re-rerun/resume after yielding the **RequestInput** response, it re-runs the dynamic child nodes using `ctx.run_node(child_node)` , which as expected, do not actually re-rerun but rehydrates the node's state using the Session Events(SSE) present in invocation context. This Events have been altered in `_consume_event_queue()`(which consume's event from the event queue) function in the runner before putting them into Session.

A Node with `message_as_input == True` (Specifically, LLMAgent node), will have its output nullified in the `_consume_event_queue()` function.

```

# _consume_event_queue() function ....
    if event.node_info.message_as_output and event.content is not None:
        event = event.model_copy()
        event.output = None
    #rest of the function.....

```



Everything good till here since, **LLMAgent** produces a event with a content object, therefore, a specific output field in the event is not required in the event.

But the main issue lies in the fact that `event.content` is not being processed before assigning it to `child.output` in `_reconstruct_node_state()` function which reconstruct the node's prior state(the state when the node previously executed), and this `child.output` is propagated as the node's output up till `ctx.run_node`.

```

#..... _reconstruct_node_state() function

    if is_direct or is_delegated:
        if event.output is not None:
            child.output = event.output
            child.branch = event.branch
        elif use_message_as_output:
            child.output = event.content
    #.....rest of the function

```



So basically, the nodes with `message_as_output == True`, are producing the expected output during fresh execution but when re-rerun and subsequently re-hydrated they produce the raw `event.content` instead of the output.

In nutshell:

1)**Fresh Run:** `run_llm_agent_as_node()` populates `event.output`. `_track_event_in_context()` sees this and correctly assigns the string/schema to `ctx.output`.

2)**Persistence:** `_consume_event_queue()` sets `event.output = None` to optimize for `message_as_output` logic and saves the raw `event.content` to the `Session`.

3)**Resumption:** When the `main_workflow` resumes, `_reconstruct_node_states()` attempts to rebuild the child state. Because `event.output` is now `None`, it falls back to:

```
elif use_message_as_output:
    child.output = event.content # <--- Error: Assigns raw container/object
```



4)**Failure:** This raw container is returned to `ctx.run_node(analyzer, report)`, causing `Action.model_validate(agent_output)` to fail because it received `types.Contentobject` instead of a valid Action schema or dictionary.

Steps to Reproduce:

Please provide a numbered list of steps to reproduce the behavior:

1. Create a main function node with LLMagent node and a node which yields RequestInput object.
2. Make the LLMagent node run before the RequestInput Node.
3. Have the LLMagent's output validated if output schema is provided or check it against the string data type.
4. The error will occur when the main function node is rerun after the getting the input from user.

Expected Behavior:

When re-running the node. LLMagent node's output should have have been the output returned during the fresh execution that is a dictionary.

Observed Behavior:

The raw `types.Content` is returned which failed to verify against the `output_schema` of the LLMagent node

Environment Details:

- ADK Library Version (pip show google-adk): 2.0.0b1
- Desktop OS:** Windows 10
- Python Version (python -V): Python 3.13.4

Model Information:

- Are you using LiteLLM: No
- Which model is being used: gemini-3.1-flash-lite-preview

Optional Information

From what I understand about the codebase, during a child node's fresh run the events are enqueued in the event queue, and event in this queue are consumed by the runner where as stated earlier they are altered.

And before the events are put in the event queue they are tracked in the context of the child node in the function `_track_event_in_context()` which puts `event.output` into child nodes `ctx.output`.

```
def _track_event_in_context(self, event: Event, ctx: Context) -> None:
    """Write yielded event results to ctx (source of truth)."""
    if event.output is not None:
        ctx.output = event.output
    elif event.node_info and event.node_info.message_as_output:
        ctx.output = event.content
    # ..... rest of the function
```



But I suppose this function intends to make the

```
ctx.output = event.content
```



instead of

```
ctx.output = event.output
```



But since it can not, because in `process_llm_agent_output()` function it puts the `event.output = output` hence, populating the event.output.

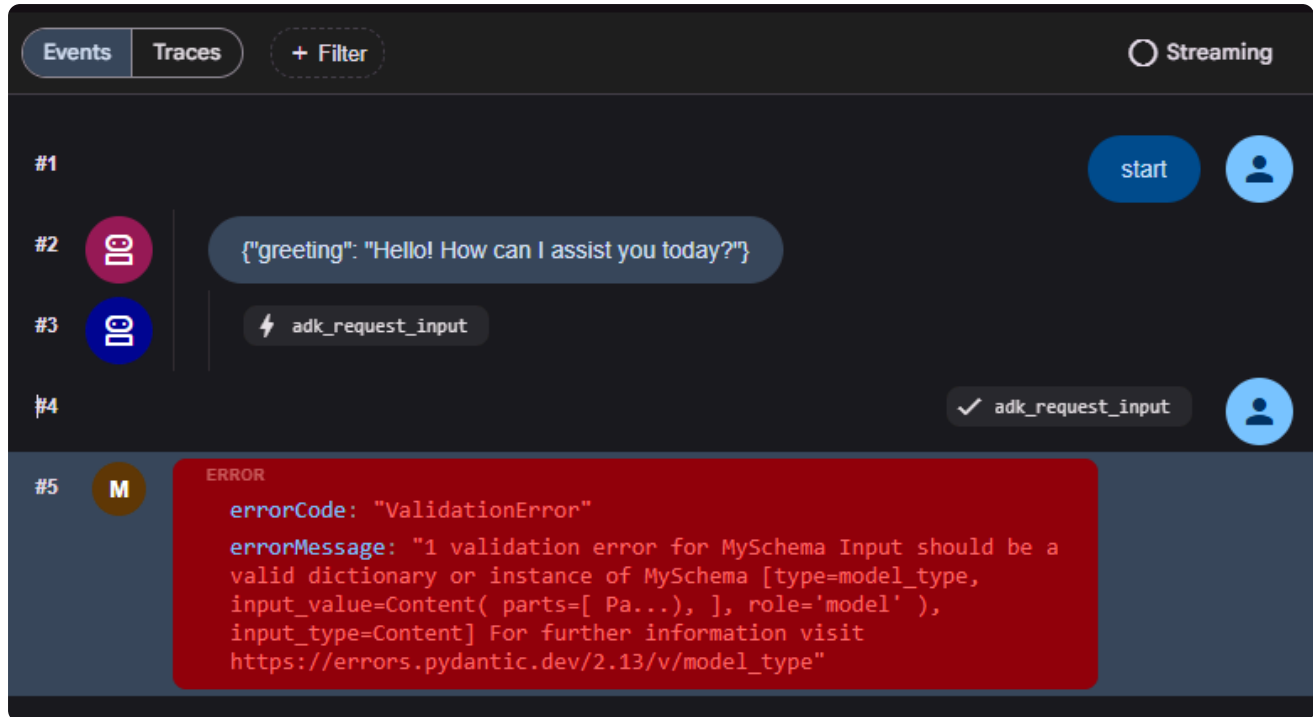
So there could additionally be one more fix by either:

1. Assign `ctx.output = event.content` (off course after processing the content, maybe using the `process_llm_agent_output()` function)

or

2. If we strictly want the output field to be None, even during the fresh execution of the node, remove `process_llm_agent_output()` function to process the output in the `run_llm_agent_as_node()` method.

Screenshots / Video:



Logs:

```

File "D:\Projects\googleadk\.venv\Lib\site-packages\google\adk\workflow\_node_runner.py", line 127, in run
    await self._execute_node(ctx, node_input)
File "D:\Projects\googleadk\.venv\Lib\site-packages\google\adk\workflow\_node_runner.py", line 250, in _execute_node
    await self._run_node_loop(ctx, node_input)
File "D:\Projects\googleadk\.venv\Lib\site-packages\google\adk\workflow\_node_runner.py", line 264, in _run_node_loop
    async for event in agen:
        self._track_event_in_context(event, ctx)
        await self._enqueue_event(event, ctx)
File "D:\Projects\googleadk\.venv\Lib\site-packages\google\adk\workflow\_base_node.py", line 218, in run
    async for item in agen:
        ...<12 lines>...
        yield Event(output=validated)
File "D:\Projects\googleadk\.venv\Lib\site-packages\google\adk\workflow\_function_node.py", line 508, in _run_impl
    async for item in items:
        ...<2 lines>...
        yield event
File "D:\Projects\googleadk\agent\agent.py", line 78, in main_workflow
    agent_output = Action.model_validate(agent_output)
File "D:\Projects\googleadk\.venv\Lib\site-packages\pydantic\main.py", line 732, in model_validate
    return cls.__pydantic_validator__.validate_python(
        ~~~~~^
        obj,
        ^^^^
    ...<5 lines>...
        by_name=by_name,
        ^^^^^^^^^^^^^^^^^
    )
    ^

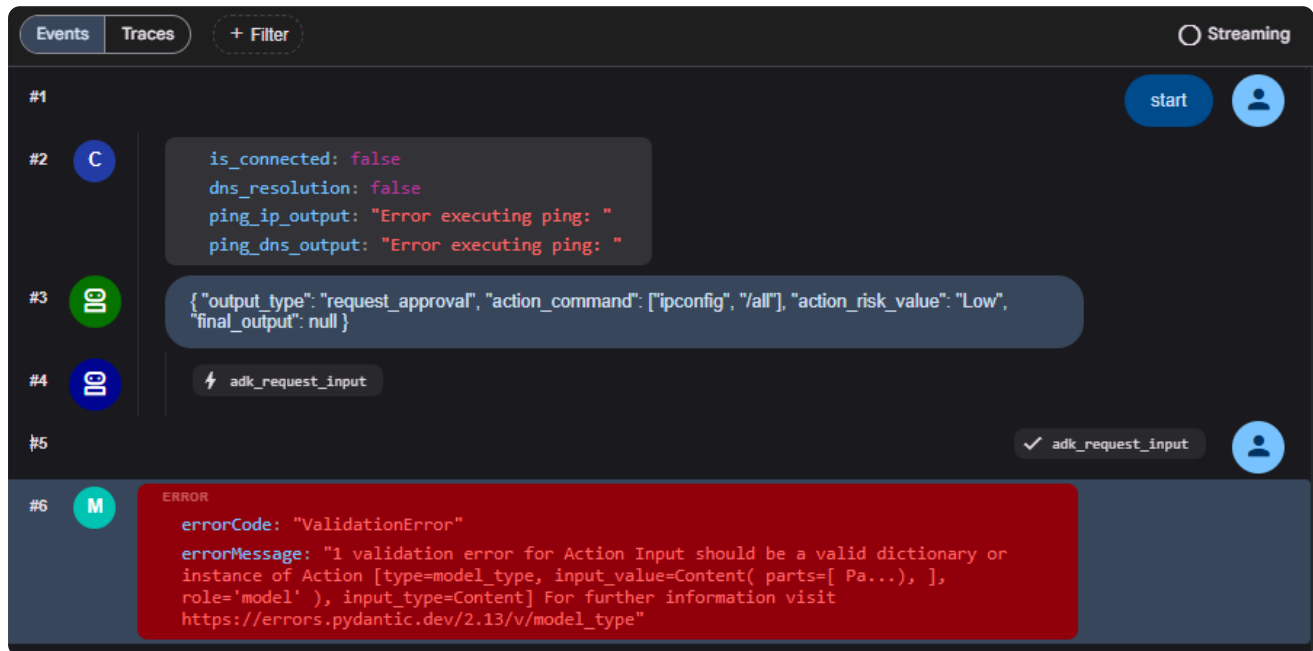
```

```

pydantic_core._pydantic_core.ValidationError: 1 validation error for Action
  Input should be a valid dictionary or instance of Action [type=model_type,
input_value=Content(
  parts=[
    Pa...),
  ],
  role='model'
), input_type=Content]

```

Screenshots / Video:



Additional Context:

'N/A

Minimal Reproduction Code:

Please provide a code snippet or a link to a Gist/repo that isolates the issue.

```

from google.adk import Workflow
from google.adk.events import RequestInput
from google.adk import Context
from google.adk.agents import Agent
from google.adk.workflow import node

from pydantic import BaseModel

from typing import Any
class MySchema(BaseModel):
    greeting: str

analyzer = Agent(
    name="my_agent",
    model="gemini-3.1-flash-lite-preview",
    output_schema=MySchema,
    instruction="Your job is to greet the user.",
)
@node(rerun_on_resume=False)
async def get_user_approval(ctx: Context, node_input: Any):
    """Yields a RequestInput to pause the workflow and wait for user input."""

```



```

yield RequestInput(message=f'please approve or reject.', response_schema = str)

@node(rerun_on_resume=True)
async def handle_process(ctx: Context, node_input: Any):
    """The orchestrator calling the interactive step."""
    user_response = await ctx.run_node(get_user_approval, node_input)
    if user_response.lower() == "yes":
        yield 'approved'
    yield "Denied"
    return

@node(rerun_on_resume=True)
async def my_workflow(ctx: Context):
    agent_input = "my agent input"
    agent_output = await ctx.run_node(analyzer, agent_input)
    agent_output = MySchema.validate(agent_output)
    result = await ctx.run_node(handle_process)
    yield result

root_agent = Workflow(
    name="greet_user",
    edges=[("START", my_workflow)]
)

```

Screenshots / Video:

The screenshot shows the ADK Bot interface with a workflow execution. The workflow steps are:

- #1: start
- #2: {"greeting": "Hello! How can I assist you today?"}
- #3: ⚡ adk_request_input
- #4: ✓ adk_request_input
- #5: ERROR

The error message for step #5 is:

```

ERROR
errorCode: "ValidationError"
errorMessage: "1 validation error for MySchema Input should be a
valid dictionary or instance of MySchema [type=model_type,
input_value=Content( parts=[ Pa...), ], role='model' ),
input_type=Content] For further information visit
https://errors.pydantic.dev/2.13/v/model_type"

```

How often has this issue occurred?:

- Always (100%)



  **adk-bot** assigned Ilalitkumarr on Apr 30

  **adk-bot** added the **Bug** issue type on Apr 30

Closed

Raw event.content assigned to child.output without transformation in _recons... #5553



  **ValentinCordonnier** mentioned this on May 5

 fix(workflow): preserve event.output for cached LlmAgent results #5587

 **Ilalitkumarr** on May 5

Collaborator



Hello @samarth1224,

Thank you for providing such a detailed report. There is a PR linked to this issue. Could you please find some time to review it and let us know if it resolves the issue for you?



  **Ilalitkumarr** added **request clarification** on May 5

 **samarth1224** on May 5

Contributor

Author



Yes, I am happy to review the PR. I'm in the process of verifying the fix and will get back to you with a detailed review as soon as possible.



 **samarth1224** on May 5

Contributor

Author



Hi, @Ilalitkumarr,

I have submitted my review and some feedback directly on the PR.
Thanks!



  **samarth1224** mentioned this on May 12

 Fix(workflow)/ add _process_content_object function to extract output from event.content before assigning it to child.output in _reconstruct_node_states #5668

  **markns** mentioned this on May 20

 ADK 2.0.0 GA: task-mode LlmAgents disabled as workflow graph nodes (regression from 2.0.0b1) #5764



samarth1224 on May 21

Contributor

Author



Hi, @llalitikumarr just a follow up, this issue still persist in the latest v2.0 release. I have tested and confirmed it.



samarth1224 mentioned this on May 30



Fix(workflow)/add _process_content_object function in _rehydration_utils file to extract output from event.content object before assigning it to child.output, in _reconstruct_node_states #5909



DeanChensj on Jun 3

Collaborator



resolved [#5909](#)

DeanChensj closed this as completed on Jun 3

Add a comment

Write

Preview

H

B

I

≡

<>



≡

½≡

≡



Use Markdown to format your comment



Paste, drop, or click to add files



Reopen issue



Comment



Please follow this repository's [contributing guidelines](#), [security policy](#) and [code of conduct](#).

Metadata

